

Bachelorarbeit Justus Wefelscheid
Code Dokumentation

Generated by Doxygen 1.18.0

1 Namespace Index	1
1.1 Package List	1
2 Hierarchical Index	3
2.1 Class Hierarchy	3
3 Class Index	5
3.1 Class List	5
4 Namespace Documentation	7
4.1 CNN_patient_split Namespace Reference	7
4.1.1 Detailed Description	8
4.1.2 Variable Documentation	9
4.1.2.1 accuracy	9
4.1.2.2 auc	9
4.1.2.3 balanced_accuracy	9
4.1.2.4 criterion	9
4.1.2.5 DATA_DIR	9
4.1.2.6 DEVICE	10
4.1.2.7 eval_transform	10
4.1.2.8 model	10
4.1.2.9 optimizer	10
4.1.2.10 predictions	10
4.1.2.11 RESULT_DIR	11
4.1.2.12 splitter	11
4.1.2.13 test_dataset	11
4.1.2.14 test_loader	11
4.1.2.15 train_accuracy	11
4.1.2.16 train_dataset	12
4.1.2.17 train_loader	12
4.1.2.18 train_transform	12
4.1.2.19 val_accuracy	12
4.1.2.20 val_dataset	12
4.1.2.21 val_loader	13
4.2 CNN_random_split Namespace Reference	13
4.2.1 Detailed Description	14
4.2.2 Variable Documentation	15
4.2.2.1 accuracy	15
4.2.2.2 auc	15
4.2.2.3 balanced_accuracy	15
4.2.2.4 criterion	15
4.2.2.5 DATA_DIR	15
4.2.2.6 DEVICE	16

4.2.2.7 eval_transform	16
4.2.2.8 model	16
4.2.2.9 optimizer	16
4.2.2.10 predictions	16
4.2.2.11 RESULT_DIR	17
4.2.2.12 test_dataset	17
4.2.2.13 test_loader	17
4.2.2.14 train_accuracy	17
4.2.2.15 train_dataset	17
4.2.2.16 train_loader	18
4.2.2.17 train_transform	18
4.2.2.18 val_accuracy	18
4.2.2.19 val_dataset	18
4.2.2.20 val_loader	18
4.3 compare_aggregations Namespace Reference	19
4.3.1 Detailed Description	19
4.3.2 Variable Documentation	20
4.3.2.1 AGGREGATIONS	20
4.3.2.2 class_map	20
4.3.2.3 classifier	20
4.3.2.4 cv	20
4.3.2.5 FEATURE_DIRS	20
4.3.2.6 mean	21
4.3.2.7 model_features	21
4.3.2.8 model_results	21
4.3.2.9 predictions	21
4.3.2.10 repeat_predictions	21
4.3.2.11 row	22
4.3.2.12 std	22
4.4 feature_selection_aggregations Namespace Reference	22
4.4.1 Detailed Description	23
4.4.2 Function Documentation	23
4.4.2.1 load_features()	23
4.4.2.2 rank_dimensions()	24
4.4.3 Variable Documentation	25
4.4.3.1 all_features	25
4.4.3.2 class_map	25
4.4.3.3 classifier	25
4.4.3.4 FEATURE_DIRS	25
4.4.3.5 full_score	25
4.4.3.6 model_results	26
4.4.3.7 predictions	26

4.4.3.8 ranking	26
4.4.3.9 row	26
4.4.3.10 scores	26
4.4.3.11 selected	26
4.4.3.12 splits	27
4.4.3.13 splitter	27
4.4.3.14 test_3d	27
4.4.3.15 test_top_k	27
4.4.3.16 top_k_score	27
4.4.3.17 train_3d	28
4.4.3.18 train_top_k	28
4.5 feature_selection_mean Namespace Reference	28
4.5.1 Detailed Description	29
4.5.2 Function Documentation	29
4.5.2.1 load_features()	29
4.5.2.2 rank_features()	30
4.5.3 Variable Documentation	30
4.5.3.1 all_features	30
4.5.3.2 class_map	31
4.5.3.3 classifier	31
4.5.3.4 FEATURE_DIRS	31
4.5.3.5 full_balacc	31
4.5.3.6 full_f1	31
4.5.3.7 metric_names	32
4.5.3.8 oof_results	32
4.5.3.9 predictions	32
4.5.3.10 ranking	32
4.5.3.11 row	32
4.5.3.12 scores	33
4.5.3.13 selected	33
4.5.3.14 selection_counts	33
4.5.3.15 splits	33
4.5.3.16 splitter	33
4.5.3.17 summary	34
4.5.3.18 test_scaled	34
4.5.3.19 train_scaled	34
4.6 mean_baseline Namespace Reference	34
4.6.1 Detailed Description	35
4.6.2 Variable Documentation	35
4.6.2.1 class_map	35
4.6.2.2 classifier	36
4.6.2.3 cv	36

4.6.2.4 dummy_predictions	36
4.6.2.5 FEATURE_DIRS	36
4.6.2.6 model_results	36
4.6.2.7 predictions	37
4.6.2.8 repeat_predictions	37
4.6.2.9 row	37
4.6.2.10 train_balanced_accuracies	37
4.6.2.11 train_predictions	37
4.7 mean_vs_attention_MIL Namespace Reference	37
4.7.1 Detailed Description	39
4.7.2 Function Documentation	39
4.7.2.1 evaluate()	39
4.7.2.2 set_seed()	40
4.7.2.3 train_model()	40
4.7.3 Variable Documentation	41
4.7.3.1 class_map	41
4.7.3.2 cv	41
4.7.3.3 device	41
4.7.3.4 FEATURE_DIRS	42
5 Class Documentation	43
5.1 mean_vs_attention_MIL.AttentionMLP Class Reference	43
5.1.1 Detailed Description	43
5.1.2 Constructor & Destructor Documentation	44
5.1.2.1 __init__()	44
5.1.3 Member Function Documentation	44
5.1.3.1 forward()	44
5.1.4 Member Data Documentation	45
5.1.4.1 attention_u	45
5.1.4.2 attention_v	45
5.1.4.3 attention_w	45
5.1.4.4 classifier	45
5.2 CNN_patient_split.CellDataset Class Reference	46
5.2.1 Detailed Description	46
5.2.2 Constructor & Destructor Documentation	46
5.2.2.1 __init__()	46
5.2.3 Member Function Documentation	46
5.2.3.1 __getitem__()	46
5.2.3.2 __len__()	47
5.3 CNN_random_split.CellDataset Class Reference	47
5.3.1 Detailed Description	47
5.3.2 Constructor & Destructor Documentation	47

5.3.2.1 <code>__init__()</code>	47
5.3.3 Member Function Documentation	48
5.3.3.1 <code>__getitem__()</code>	48
5.3.3.2 <code>__len__()</code>	48
5.4 <code>mean_vs_attention_MIL.MeanMLP</code> Class Reference	48
5.4.1 Detailed Description	48
5.4.2 Constructor & Destructor Documentation	49
5.4.2.1 <code>__init__()</code>	49
5.4.3 Member Function Documentation	49
5.4.3.1 <code>forward()</code>	49
5.5 <code>mean_vs_attention_MIL.MLPClassifier</code> Class Reference	49
5.5.1 Detailed Description	50
5.5.2 Constructor & Destructor Documentation	50
5.5.2.1 <code>__init__()</code>	50
5.5.3 Member Function Documentation	50
5.5.3.1 <code>forward()</code>	50
5.5.4 Member Data Documentation	51
5.5.4.1 <code>layers</code>	51
Index	53

Chapter 1

Namespace Index

1.1 Package List

Here are the packages with brief descriptions (if available):

CNN_patient_split	7
CNN_random_split	13
compare_aggregations	19
feature_selection_aggregations	22
feature_selection_mean	28
mean_baseline	34
mean_vs_attention_MIL	37

Chapter 2

Hierarchical Index

2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Dataset	
CNN_patient_split.CellDataset	46
CNN_random_split.CellDataset	47
nn.Module	
mean_vs_attention_MIL.AttentionMLP	43
mean_vs_attention_MIL.MLPClassifier	49
mean_vs_attention_MIL.MeanMLP	48

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

mean_vs_attention_MIL.AttentionMLP	43
CNN_patient_split.CellDataset	46
CNN_random_split.CellDataset	47
mean_vs_attention_MIL.MeanMLP	48
mean_vs_attention_MIL.MLPClassifier	49

Chapter 4

Namespace Documentation

4.1 CNN_patient_split Namespace Reference

Classes

- class [CellDataset](#)

Variables

- [DATA_DIR](#)
- [RESULT_DIR](#)
- **parents**
- **True**
- **exist_ok**
- str **MODEL_PATH** = DATA_DIR / "best_resnet18.pth"
- int **IMAGE_SIZE** = 32
- int **BATCH_SIZE** = 32
- int **EPOCHS** = 30
- int **LEARNING_RATE** = 1e-4
- int **SEED** = 42
- [DEVICE](#)
- list **image_paths** = []
- list **labels** = []
- list **groups** = []
- **class_dir** = DATA_DIR / class_name
- **relative_path** = image_path.relative_to(class_dir)
- [splitter](#)
- **train_val_idx**
- **test_idx**
- **train_relative**
- **val_relative**
- **train_idx** = train_val_idx[train_relative]
- **val_idx** = train_val_idx[val_relative]
- [train_transform](#)
- [eval_transform](#)
- [train_dataset](#)
- [val_dataset](#)

- [test_dataset](#)
- [train_loader](#)
- [val_loader](#)
- [test_loader](#)
- [model](#)
- **fc**
- **number_negative** = np.sum(labels[train_idx] == 0)
- **number_positive** = np.sum(labels[train_idx] == 1)
- **positive_weight** = number_negative / number_positive
- [criterion](#)
- [optimizer](#)
- **best_val_loss** = float("inf")
- int **patience** = 7
- int **epochs_without_improvement** = 0
- list **train_losses** = []
- list **val_losses** = []
- list **train_accuracies** = []
- list **val_accuracies** = []
- int **train_loss** = 0
- list **train_predictions** = []
- list **train_targets** = []
- **images** = images.to(DEVICE)
- **targets** = targets.to(DEVICE)
- **logits** = model(images).squeeze(1)
- **loss** = criterion(logits, targets)
- tuple [predictions](#)
- [train_accuracy](#)
- int **val_loss** = 0
- list **val_predictions** = []
- list **val_targets** = []
- [val_accuracy](#)
- **epochs** = range(1, len(train_losses) + 1)
- **label**
- **dpi**
- list **test_probabilities** = []
- list **test_predictions** = []
- list **test_targets** = []
- **probabilities** = torch.sigmoid(logits)
- [accuracy](#)
- [balanced_accuracy](#)
- [auc](#)

4.1.1 Detailed Description

Trainiert eine ResNet-18 zur binären Ki-67-Zellklassifikation.

Die Zellbilder werden slide-basiert in Trainings-, Validierungs- und Testdaten aufgeteilt, sodass Zellen desselben Slides nicht in mehreren Datensätzen vorkommen. Das Modell wird mit gewichteter binärer Kreuzentropie trainiert.

Während des Trainings werden Loss und Balanced Accuracy für Training und Validierung aufgezeichnet. Das Modell mit dem niedrigsten Validation Loss wird gespeichert und anschließend auf dem Testdatensatz ausgewertet.

4.1.2 Variable Documentation

4.1.2.1 accuracy

CNN_patient_split.accuracy

Initial value:

```
= accuracy_score(  
    test_targets,  
    test_predictions,  
)
```

4.1.2.2 auc

CNN_patient_split.auc

Initial value:

```
= roc_auc_score(  
    test_targets,  
    test_probabilities,  
)
```

4.1.2.3 balanced_accuracy

CNN_patient_split.balanced_accuracy

Initial value:

```
= balanced_accuracy_score(  
    test_targets,  
    test_predictions,  
)
```

4.1.2.4 criterion

CNN_patient_split.criterion

Initial value:

```
= nn.BCEWithLogitsLoss(  
    pos_weight=torch.tensor(  
        [positive_weight],  
        dtype=torch.float32,  
        device=DEVICE,  
    )  
)
```

4.1.2.5 DATA_DIR

CNN_patient_split.DATA_DIR

Initial value:

```
= Path(  
    "/Users/justuswefelscheid/Documents/Studium/"  
    "Bachelorarbeit/ki67_QuPath_Project/cell_patches"  
)
```

4.1.2.6 DEVICE

CNN_patient_split.DEVICE

Initial value:

```
= torch.device(  
    "mps" if torch.backends.mps.is_available()  
    else "cuda" if torch.cuda.is_available()  
    else "cpu"  
)
```

4.1.2.7 eval_transform

CNN_patient_split.eval_transform

Initial value:

```
= transforms.Compose([  
    transforms.Resize((IMAGE_SIZE, IMAGE_SIZE)),  
    transforms.Grayscale(num_output_channels=3),  
    transforms.ToTensor(),  
    transforms.Normalize(  
        mean=[0.485, 0.456, 0.406],  
        std=[0.229, 0.224, 0.225],  
    ),  
)
```

4.1.2.8 model

CNN_patient_split.model

Initial value:

```
= models.resnet18(  
    weights=models.ResNet18_Weights.DEFAULT  
)
```

4.1.2.9 optimizer

CNN_patient_split.optimizer

Initial value:

```
= torch.optim.Adam(  
    model.parameters(),  
    lr=LEARNING_RATE,  
)
```

4.1.2.10 predictions

tuple predictions

Initial value:

```
= (  
    torch.sigmoid(logits) >= 0.5  
) .int()
```

4.1.2.11 RESULT_DIR

CNN_patient_split.RESULT_DIR

Initial value:

```
= Path(  
    "/Users/justuswefelscheid/Documents/Studium/"  
    "Bachelorarbeit/Ki67_Pipeline/results"  
)
```

4.1.2.12 splitter

CNN_patient_split.splitter

Initial value:

```
= GroupShuffleSplit(  
    n_splits=1,  
    test_size=0.20,  
    random_state=SEED,  
)
```

4.1.2.13 test_dataset

CNN_patient_split.test_dataset

Initial value:

```
= CellDataset(  
    image_paths[test_idx],  
    labels[test_idx],  
    eval_transform,  
)
```

4.1.2.14 test_loader

CNN_patient_split.test_loader

Initial value:

```
= DataLoader(  
    test_dataset,  
    batch_size=BATCH_SIZE,  
    shuffle=False,  
    num_workers=0,  
)
```

4.1.2.15 train_accuracy

CNN_patient_split.train_accuracy

Initial value:

```
= balanced_accuracy_score(  
    train_targets,  
    train_predictions,  
)
```

4.1.2.16 train_dataset

CNN_patient_split.train_dataset

Initial value:

```
= CellDataset(  
    image_paths[train_idx],  
    labels[train_idx],  
    train_transform,  
)
```

4.1.2.17 train_loader

CNN_patient_split.train_loader

Initial value:

```
= DataLoader(  
    train_dataset,  
    batch_size=BATCH_SIZE,  
    shuffle=True,  
    num_workers=0,  
)
```

4.1.2.18 train_transform

CNN_patient_split.train_transform

Initial value:

```
= transforms.Compose([  
    transforms.Resize((IMAGE_SIZE, IMAGE_SIZE)),  
    transforms.Grayscale(num_output_channels=3),  
    transforms.RandomHorizontalFlip(),  
    transforms.RandomVerticalFlip(),  
    transforms.RandomRotation(180),  
    transforms.ToTensor(),  
    transforms.Normalize(  
        mean=[0.485, 0.456, 0.406],  
        std=[0.229, 0.224, 0.225],  
    ),  
)
```

4.1.2.19 val_accuracy

CNN_patient_split.val_accuracy

Initial value:

```
= balanced_accuracy_score(  
    val_targets,  
    val_predictions,  
)
```

4.1.2.20 val_dataset

CNN_patient_split.val_dataset

Initial value:

```
= CellDataset(  
    image_paths[val_idx],  
    labels[val_idx],  
    eval_transform,  
)
```

4.1.2.21 val_loader

CNN_patient_split.val_loader

Initial value:

```
= DataLoader(
    val_dataset,
    batch_size=BATCH_SIZE,
    shuffle=False,
    num_workers=0,
)
```

4.2 CNN_random_split Namespace Reference

Classes

- class [CellDataset](#)

Variables

- [DATA_DIR](#)
- [RESULT_DIR](#)
- **parents**
- **True**
- **exist_ok**
- str **MODEL_PATH** = RESULT_DIR / "best_resnet18_random_split.pth"
- int **IMAGE_SIZE** = 32
- int **BATCH_SIZE** = 32
- int **EPOCHS** = 30
- int **LEARNING_RATE** = 1e-4
- int **SEED** = 42
- [DEVICE](#)
- list **image_paths** = []
- list **labels** = []
- **class_dir** = DATA_DIR / class_name
- **all_indices** = np.arange(len(image_paths))
- **train_val_idx**
- **test_idx**
- **test_size**
- **random_state**
- **stratify**
- **train_idx**
- **val_idx**
- [train_transform](#)
- [eval_transform](#)
- [train_dataset](#)
- [val_dataset](#)
- [test_dataset](#)
- [train_loader](#)
- [val_loader](#)
- [test_loader](#)
- [model](#)
- **fc**

- **number_negative** = np.sum(labels[train_idx] == 0)
- **number_positive** = np.sum(labels[train_idx] == 1)
- **positive_weight** = number_negative / number_positive
- [criterion](#)
- [optimizer](#)
- **best_val_loss** = float("inf")
- int **patience** = 7
- int **epochs_without_improvement** = 0
- list **train_losses** = []
- list **val_losses** = []
- list **train_accuracies** = []
- list **val_accuracies** = []
- int **train_loss** = 0
- list **train_predictions** = []
- list **train_targets** = []
- **images** = images.to(DEVICE)
- **targets** = targets.to(DEVICE)
- **logits** = model(images).squeeze(1)
- **loss** = criterion(logits, targets)
- tuple [predictions](#)
- [train_accuracy](#)
- int **val_loss** = 0
- list **val_predictions** = []
- list **val_targets** = []
- [val_accuracy](#)
- **epochs** = range(1, len(train_losses) + 1)
- **label**
- **dpi**
- list **test_probabilities** = []
- list **test_predictions** = []
- list **test_targets** = []
- **probabilities** = torch.sigmoid(logits)
- [accuracy](#)
- [balanced_accuracy](#)
- [auc](#)

4.2.1 Detailed Description

Trainiert eine ResNet-18 zur binären Ki-67-Zellklassifikation.

Die Zellbilder werden zufällig und klassenstratifiziert in Trainings-, Validierungs- und Testdaten aufgeteilt. Dadurch können Zellen desselben Slides in mehreren Datensätzen vorkommen. Das Modell wird mit gewichteter binärer Kreuzentropie trainiert.

Während des Trainings werden Loss und Balanced Accuracy für Training und Validierung aufgezeichnet. Das Modell mit dem niedrigsten Validation Loss wird gespeichert und anschließend auf dem Testdatensatz ausgewertet.

4.2.2 Variable Documentation

4.2.2.1 accuracy

CNN_random_split.accuracy

Initial value:

```
= accuracy_score(  
    test_targets,  
    test_predictions,  
)
```

4.2.2.2 auc

CNN_random_split.auc

Initial value:

```
= roc_auc_score(  
    test_targets,  
    test_probabilities,  
)
```

4.2.2.3 balanced_accuracy

CNN_random_split.balanced_accuracy

Initial value:

```
= balanced_accuracy_score(  
    test_targets,  
    test_predictions,  
)
```

4.2.2.4 criterion

CNN_random_split.criterion

Initial value:

```
= nn.BCEWithLogitsLoss(  
    pos_weight=torch.tensor(  
        [positive_weight],  
        dtype=torch.float32,  
        device=DEVICE,  
    )  
)
```

4.2.2.5 DATA_DIR

CNN_random_split.DATA_DIR

Initial value:

```
= Path(  
    "/Users/justuswefelscheid/Documents/Studium/"  
    "Bachelorarbeit/ki67_QuPath_Project/cell_patches"  
)
```

4.2.2.6 DEVICE

CNN_random_split.DEVICE

Initial value:

```
= torch.device(  
    "mps" if torch.backends.mps.is_available()  
    else "cuda" if torch.cuda.is_available()  
    else "cpu"  
)
```

4.2.2.7 eval_transform

CNN_random_split.eval_transform

Initial value:

```
= transforms.Compose([  
    transforms.Resize((IMAGE_SIZE, IMAGE_SIZE)),  
    transforms.ToTensor(),  
    transforms.Normalize(  
        mean=[0.485, 0.456, 0.406],  
        std=[0.229, 0.224, 0.225],  
    ),  
)
```

4.2.2.8 model

CNN_random_split.model

Initial value:

```
= models.resnet18(  
    weights=models.ResNet18_Weights.DEFAULT  
)
```

4.2.2.9 optimizer

CNN_random_split.optimizer

Initial value:

```
= torch.optim.Adam(  
    model.parameters(),  
    lr=LEARNING_RATE,  
)
```

4.2.2.10 predictions

tuple predictions

Initial value:

```
= (  
    torch.sigmoid(logits) >= 0.5  
) .int()
```

4.2.2.11 RESULT_DIR

CNN_random_split.RESULT_DIR

Initial value:

```
= Path(  
    "/Users/justuswefelscheid/Documents/Studium/"  
    "Bachelorarbeit/Ki67_Pipeline/results"  
)
```

4.2.2.12 test_dataset

CNN_random_split.test_dataset

Initial value:

```
= CellDataset(  
    image_paths[test_idx],  
    labels[test_idx],  
    eval_transform,  
)
```

4.2.2.13 test_loader

CNN_random_split.test_loader

Initial value:

```
= DataLoader(  
    test_dataset,  
    batch_size=BATCH_SIZE,  
    shuffle=False,  
    num_workers=0,  
)
```

4.2.2.14 train_accuracy

CNN_random_split.train_accuracy

Initial value:

```
= balanced_accuracy_score(  
    train_targets,  
    train_predictions,  
)
```

4.2.2.15 train_dataset

CNN_random_split.train_dataset

Initial value:

```
= CellDataset(  
    image_paths[train_idx],  
    labels[train_idx],  
    train_transform,  
)
```

4.2.2.16 train_loader

CNN_random_split.train_loader

Initial value:

```
= DataLoader(  
    train_dataset,  
    batch_size=BATCH_SIZE,  
    shuffle=True,  
    num_workers=0,  
)
```

4.2.2.17 train_transform

CNN_random_split.train_transform

Initial value:

```
= transforms.Compose([  
    transforms.Resize((IMAGE_SIZE, IMAGE_SIZE)),  
    transforms.RandomHorizontalFlip(),  
    transforms.RandomVerticalFlip(),  
    transforms.RandomRotation(180),  
    transforms.ColorJitter(  
        brightness=0.1,  
        contrast=0.1,  
        saturation=0.1,  
    ),  
    transforms.ToTensor(),  
    transforms.Normalize(  
        mean=[0.485, 0.456, 0.406],  
        std=[0.229, 0.224, 0.225],  
    ),  
)
```

4.2.2.18 val_accuracy

CNN_random_split.val_accuracy

Initial value:

```
= balanced_accuracy_score(  
    val_targets,  
    val_predictions,  
)
```

4.2.2.19 val_dataset

CNN_random_split.val_dataset

Initial value:

```
= CellDataset(  
    image_paths[val_idx],  
    labels[val_idx],  
    eval_transform,  
)
```

4.2.2.20 val_loader

CNN_random_split.val_loader

Initial value:

```
= DataLoader(  
    val_dataset,  
    batch_size=BATCH_SIZE,  
    shuffle=False,  
    num_workers=0,  
)
```

4.3 compare_aggregations Namespace Reference

Variables

- **ROOT** = Path("/home/ubuntu/applications/ki67-thesis")
- str **LABEL_FILE** = ROOT / "data/labels/ki67_clean_20_HE.csv"
- str **RESULT_DIR** = ROOT / "results/aggregation_comparison"
- **parents**
- **True**
- **exist_ok**
- dict **FEATURE_DIRS**
- list **AGGREGATIONS**
- int **N_SPLITS** = 5
- int **N_REPEATS** = 50
- int **RANDOM_STATE** = 42
- **labels** = pd.read_csv(LABEL_FILE)
- dict **class_map**
- **y** = labels["ki67_class"].map(class_map).to_numpy().astype(int)
- **cv**
- **splits** = list(cv.split(np.zeros(len(y)), y))
- dict **slide_features** = {}
- dict **model_features**
- str **h5_path** = feature_dir / f"{Path(he_id).stem}.h5"
- **patch_features** = f["features"][:]
- **mean**
- **std**
- **q25**
- **median**
- **q75**
- **axis**
- list **repeat_rows** = []
- list **oof_frames** = []
- dict **X** = slide_features[(model_name, agg_name)]
- list **repeat_predictions**
- int **repeat** = split_number // N_SPLITS
- **classifier**
- **predictions**
- **ignore_index**
- **index**
- **repeat_df** = pd.DataFrame(repeat_rows)
- list **summary_rows** = []
- **model_results**
- dict **row**
- **values** = model_results[metric].to_numpy()
- **ddof**
- **summary** = pd.DataFrame(summary_rows)

4.3.1 Detailed Description

Vergleicht vier verschiedene Aggregationsmethoden, um die Patch-Repräsentationen zusammenzufassen.

Pro Gesamtbild werden entweder nur Durchschnitte oder zusätzliche Verteilungsmerkmale zur Verteilung der Merkmale genutzt. Drei Modelle liefern die Bildmerkmale. Für jede Kombination wird die Vorhersage der Ki-67-Klasse in 50 Durchläufen geprüft. Jeder Durchlauf teilt die Bilder in fünf Gruppen: Vier dienen zum Lernen, die fünfte zum Prüfen. Jede Gruppe wird einmal zum Prüfen verwendet.

4.3.2 Variable Documentation

4.3.2.1 AGGREGATIONS

list compare_aggregations.AGGREGATIONS

Initial value:

```
= [
    "mean",
    "mean_std",
    "mean_std_median",
    "mean_std_quantiles",
]
```

4.3.2.2 class_map

dict compare_aggregations.class_map

Initial value:

```
= {
    "low": 0,
    "intermediate": 1,
    "high": 2,
}
```

4.3.2.3 classifier

compare_aggregations.classifier

Initial value:

```
= Pipeline([
    ("scaler", StandardScaler()),
    (
        "classifier",
        OneVsRestClassifier(LogisticRegression(
            C=1.0,
            solver="liblinear",
            dual=True,
            max_iter=2000,
        )),
    ),
])
```

4.3.2.4 cv

compare_aggregations.cv

Initial value:

```
= RepeatedStratifiedKFold(
    n_splits=N_SPLITS,
    n_repeats=N_REPEATS,
    random_state=RANDOM_STATE,
)
```

4.3.2.5 FEATURE_DIRS

dict compare_aggregations.FEATURE_DIRS

Initial value:

```
= {
    "gigapath": ROOT / "trident_runs/HE_3models/20x_256px_0px_overlap/features_gigapath",
    "uni_v2": ROOT / "trident_runs/HE_3models/20x_256px_0px_overlap/features_uni_v2",
    "virchow2": ROOT / "trident_runs/HE_3models/20x_224px_0px_overlap/features_virchow2",
}
```

4.3.2.6 mean

`compare_aggregations.mean`

Initial value:

```
= np.mean(  
    patch_features,  
    axis=0,  
    dtype=np.float64,  
)
```

4.3.2.7 model_features

`dict compare_aggregations.model_features`

Initial value:

```
= {  
    agg_name: []  
    for agg_name in AGGREGATIONS  
}
```

4.3.2.8 model_results

`compare_aggregations.model_results`

Initial value:

```
= repeat_df[  
    (repeat_df["model"] == model_name)  
    & (repeat_df["aggregation"] == agg_name)  
]
```

4.3.2.9 predictions

`list predictions`

Initial value:

```
= classifier.predict(  
    X[test_idx]  
)
```

4.3.2.10 repeat_predictions

`list compare_aggregations.repeat_predictions`

Initial value:

```
= [  
    np.zeros(len(y), dtype=int)  
    for _ in range(N_REPEATS)  
]
```

4.3.2.11 row

```
dict compare_aggregations.row
```

Initial value:

```
= {
    "model": model_name,
    "aggregation": agg_name,
    "feature_dim": int(
        model_results["feature_dim"].iloc[0]
    ),
}
```

4.3.2.12 std

```
compare_aggregations.std
```

Initial value:

```
= np.std(
    patch_features,
    axis=0,
    dtype=np.float64,
)
```

4.4 feature_selection_aggregations Namespace Reference

Functions

- [load_features](#) (feature_dir)
- [rank_dimensions](#) (X, y_train)

Variables

- **ROOT** = Path("/home/ubuntu/applications/ki67-thesis")
- str **LABEL_FILE** = ROOT / "data/labels/ki67_clean_20_HE.csv"
- str **OUT_DIR** = ROOT / "results/feature_selection_aggregations"
- **parents**
- **True**
- **exist_ok**
- dict [FEATURE_DIRS](#)
- list **K_VALUES** = [20, 50, 100, 200, 500, 1000]
- int **N_SPLITS** = 5
- int **N_REPEATS** = 50
- int **N_STATS** = 5
- **labels** = pd.read_csv(LABEL_FILE)
- **regex**
- dict [class_map](#)
- **y** = labels["ki67_class"].map(class_map).to_numpy()
- dict [all_features](#)
- [splitter](#)
- [splits](#)
- list **results** = []
- list **oof_frames** = []
- **start**

- tuple **repeat** = (split_number - 1) // N_SPLITS + 1
- tuple **fold** = (split_number - 1) % N_SPLITS + 1
- **y_train** = y[train_idx]
- **y_test** = y[test_idx]
- **train_full** = X[train_idx].reshape(len(train_idx), -1)
- **test_full** = X[test_idx].reshape(len(test_idx), -1)
- **scaler** = StandardScaler()
- **train_scaled** = scaler.fit_transform(train_full)
- **test_scaled** = scaler.transform(test_full)
- [train_3d](#)
- [test_3d](#)
- [classifier](#)
- [predictions](#)
- [full_score](#)
- [ranking](#)
- **top_k** = ranking[:k]
- [train_top_k](#)
- [test_top_k](#)
- [top_k_score](#)
- **oof_df** = pd.concat(oof_frames, ignore_index=True)
- **index**
- list **repeat_rows** = []
- **full** = model_oof[model_oof["variant"] == "full"]
- [selected](#)
- dict **row**
- **repeat_df** = pd.DataFrame(repeat_rows)
- [model_results](#)
- [scores](#)

4.4.1 Detailed Description

Prüft, wie gut eine Reduktion der Features die Ki-67-Klasse vorhersagt.

Für jedes Slide werden die Features der Patches durch fünf Werte zusammengefasst: Durchschnitt, Streuung, mittlerer Wert sowie die Werte bei 25 und 75 Prozent der sortierten Daten. Verglichen werden alle Merkmale und unterschiedlich große Auswahlen. Zu jedem ausgewählten Merkmal bleiben alle fünf Werte erhalten. In 50 Durchläufen wird jedes Bild genau einmal geprüft. Jeweils vier von fünf Gruppen dienen zum Lernen, eine zum Prüfen. Die einzelnen Vorhersagen stehen in ``oof\_predictions.csv``, die Kennzahlen aus allen Bildern je Durchlauf in ``repeat\_metrics.csv``.

Die Konsole zeigt die Balanced Accuracy: den Durchschnitt der Trefferquoten der einzelnen Klassen. Dadurch zählt jede Klasse gleich viel.

4.4.2 Function Documentation

4.4.2.1 load_features()

```
load_features (
    feature_dir)
```

Fasst die Bildausschnitte jedes Bildes durch fünf Werte je Merkmal zusammen.

```

00059 def load_features(feature_dir):
00060     """
00061     Fasst die Bildausschnitte jedes Bildes durch fünf Werte je Merkmal zusammen.
00062     """
00063
00064     slide_features = []
00065
00066     for he_id in labels["he_id"]:
00067
00068         with h5py.File(feature_dir / f"{he_id}.h5", "r") as f:
00069             features = f["features"][:]
00070
00071             mean = np.mean(features, axis=0)
00072             std = np.std(features, axis=0)
00073             median = np.median(features, axis=0)
00074             q25 = np.quantile(features, 0.25, axis=0)
00075             q75 = np.quantile(features, 0.75, axis=0)
00076
00077             slide_features.append(
00078                 np.stack([
00079                     mean,
00080                     std,
00081                     median,
00082                     q25,
00083                     q75,
00084                 ])
00085             )
00086
00087     return np.array(slide_features)
00088
00089

```

4.4.2.2 rank_dimensions()

```

rank_dimensions (
    X,
    y_train)

```

Sortiert Merkmale nach der Stärke ihrer gelernten Modellgewichte.

```

00090 def rank_dimensions(X, y_train):
00091     """Sortiert Merkmale nach der Stärke ihrer gelernten Modellgewichte.
00092     """
00093
00094     n_dims = X.shape[2]
00095
00096     classifier = OneVsRestClassifier(LogisticRegression(
00097         C=1.0,
00098         solver="liblinear",
00099         max_iter=2000,
00100     ))
00101
00102     classifier.fit(
00103         X.reshape(len(X), -1),
00104         y_train,
00105     )
00106
00107     # Die Gewichte der einzelnen Klassenmodelle werden gemeinsam ausgewertet.
00108     coefficients = np.vstack([model.coef_ for model in classifier.estimators_])
00109     coefficients = coefficients.reshape(
00110         len(classifier.estimators_),
00111         N_STATS,
00112         n_dims,
00113     )
00114
00115     # Quadrieren verhindert, dass sich positive und negative Gewichte aufheben.
00116     importance = np.sqrt(
00117         np.sum(coefficients ** 2, axis=(0, 1))
00118     )
00119
00120     return np.argsort(importance)[::-1]
00121
00122

```

4.4.3 Variable Documentation

4.4.3.1 all_features

dict feature_selection_aggregations.all_features

Initial value:

```
= {
    model_name: load_features(feature_dir)
    for model_name, feature_dir in FEATURE_DIRS.items()
}
```

4.4.3.2 class_map

dict feature_selection_aggregations.class_map

Initial value:

```
= {
    "low": 0,
    "intermediate": 1,
    "high": 2,
}
```

4.4.3.3 classifier

feature_selection_aggregations.classifier

Initial value:

```
= OneVsRestClassifier(LogisticRegression(
    C=1.0,
    solver="liblinear",
    max_iter=2000,
))
```

4.4.3.4 FEATURE_DIRS

dict feature_selection_aggregations.FEATURE_DIRS

Initial value:

```
= {
    "gigapath": ROOT / "trident_runs/HE_3models/20x_256px_0px_overlap/features_gigapath",
    "uni2": ROOT / "trident_runs/HE_3models/20x_256px_0px_overlap/features_uni_v2",
    "virchow2": ROOT / "trident_runs/HE_3models/20x_224px_0px_overlap/features_virchow2",
}
```

4.4.3.5 full_score

feature_selection_aggregations.full_score

Initial value:

```
= balanced_accuracy_score(
    y_test,
    predictions,
)
```

4.4.3.6 model_results

`feature_selection_aggregations.model_results`

Initial value:

```
= repeat_df[
    repeat_df["model"] == model_name
]
```

4.4.3.7 predictions

`feature_selection_aggregations.predictions`

Initial value:

```
= classifier.predict(
    test_scaled
)
```

4.4.3.8 ranking

`feature_selection_aggregations.ranking`

Initial value:

```
= rank_dimensions(
    train_3d,
    y_train,
)
```

4.4.3.9 row

`dict feature_selection_aggregations.row`

Initial value:

```
= {
    "model": model_name,
    "repeat": repeat,
    "k": k,
    "full": balanced_accuracy_score(full["y_true"], full["y_pred"]),
    "top_k": balanced_accuracy_score(selected["y_true"], selected["y_pred"]),
}
```

4.4.3.10 scores

`feature_selection_aggregations.scores`

Initial value:

```
= model_results[
    model_results["k"] == k
]["top_k"]
```

4.4.3.11 selected

`feature_selection_aggregations.selected`

Initial value:

```
= model_oof[
    (model_oof["variant"] == "top_k") & (model_oof["k"] == k)
]
```

4.4.3.12 splits

feature_selection_aggregations.splits

Initial value:

```
= list(
    splitter.split(
        np.zeros(len(y)),
        y,
    )
)
```

4.4.3.13 splitter

feature_selection_aggregations.splitter

Initial value:

```
= RepeatedStratifiedKFold(
    n_splits=N_SPLITS,
    n_repeats=N_REPEATS,
    random_state=42,
)
```

4.4.3.14 test_3d

feature_selection_aggregations.test_3d

Initial value:

```
= test_scaled.reshape(
    len(test_idx),
    N_STATS,
    X.shape[2],
)
```

4.4.3.15 test_top_k

feature_selection_aggregations.test_top_k

Initial value:

```
= test_3d[:, :, top_k].reshape(
    len(test_idx),
    -1,
)
```

4.4.3.16 top_k_score

feature_selection_aggregations.top_k_score

Initial value:

```
= balanced_accuracy_score(
    y_test,
    predictions,
)
```

4.4.3.17 train_3d

`feature_selection_aggregations.train_3d`

Initial value:

```
= train_scaled.reshape(
    len(train_idx),
    N_STATS,
    X.shape[2],
)
```

4.4.3.18 train_top_k

`feature_selection_aggregations.train_top_k`

Initial value:

```
= train_3d[:, :, top_k].reshape(
    len(train_idx),
    -1,
)
```

4.5 feature_selection_mean Namespace Reference

Functions

- [load_features](#) (feature_dir)
- [rank_features](#) (X_train, y_train)

Variables

- **ROOT** = Path("/home/ubuntu/applications/ki67-thesis")
- str **LABEL_FILE** = ROOT / "data/labels/ki67_clean_20_HE.csv"
- str **OUT_DIR** = ROOT / "results/feature_selection_multiple_k"
- **parents**
- **True**
- **exist_ok**
- dict [FEATURE_DIRS](#)
- list **K_VALUES** = [20, 50, 100, 200, 500, 1000, 1500]
- int **N_SPLITS** = 5
- int **N_REPEATS** = 50
- **labels** = pd.read_csv(LABEL_FILE)
- **regex**
- dict [class_map](#)
- **y** = labels["ki67_class"].map(class_map).to_numpy()
- dict [all_features](#)
- dict [selection_counts](#)
- [splitter](#)
- [splits](#)
- list **results** = []
- list **oof_frames** = []
- **start**
- tuple **repeat** = (split_number - 1) // N_SPLITS + 1
- tuple **fold** = (split_number - 1) % N_SPLITS + 1

- **y_train** = y[train_idx]
- **y_test** = y[test_idx]
- dict **prepared** = {}
- **scaler** = StandardScaler()
- **train_scaled**
- **test_scaled**
- **classifier**
- **predictions**
- **full_balacc**
- **full_f1**
- **ranking**
- dict **row**
- dict **top_k** = prepared[model_name]["ranking"][:k]
- **oof_df** = pd.concat(oof_frames, ignore_index=True)
- **index**
- list **repeat_rows** = []
- **full** = model_oof[model_oof["variant"] == "full"]
- **selected**
- **average**
- **zero_division**
- **repeat_df** = pd.DataFrame(repeat_rows)
- **oof_results**
- dict **metric_names**
- **columns**
- tuple **summary**
- list **frequency_rows** = []
- **frequency** = pd.DataFrame(frequency_rows)
- **scores**

4.5.1 Detailed Description

Vergleicht die Feature-Selektion für mean-gepoolte Slide-Features.

Die Patch-Features jedes Slides werden durch Mean Pooling zu einem Slide-Level-Featurevektor zusammengefasst. Für GigaPath, UNI2-h und Virchow2 wird untersucht, wie sich unterschiedliche Anzahlen ausgewählter Features gegenüber der Verwendung aller Features verhalten.

Die Auswertung erfolgt mit 50 Wiederholungen einer stratifizierten 5-Fold-Cross-Validation. Innerhalb jeder Wiederholung wird jedes Slide genau einmal als Testfall verwendet. Feature-Ranking und Standardisierung werden ausschließlich anhand der jeweiligen Trainingsdaten bestimmt.

Die Out-of-Fold-Vorhersagen werden zusätzlich gespeichert. Balanced Accuracy mittelt die klassenspezifischen Trefferquoten, Macro F1 den F1-Score über alle Klassen mit gleicher Gewichtung.

4.5.2 Function Documentation

4.5.2.1 load_features()

```
load_features (
    feature_dir)
```

Berechnet durch Mean Pooling einen Featurevektor pro Slide.

```

00060 def load_features(feature_dir):
00061     """Berechnet durch Mean Pooling einen Featurevektor pro Slide."""
00062
00063     slide_features = []
00064
00065     for he_id in labels["he_id"]:
00066
00067         with h5py.File(feature_dir / f"{he_id}.h5", "r") as f:
00068             patch_features = f["features"][:]
00069
00070             slide_features.append(
00071                 patch_features.mean(axis=0)
00072             )
00073
00074     return np.array(slide_features)
00075
00076

```

4.5.2.2 rank_features()

```

rank_features (
    X_train,
    y_train)

```

Sortiert Features nach der Stärke ihrer One-vs-Rest-Koeffizienten.

```

00077 def rank_features(X_train, y_train):
00078     """Sortiert Features nach der Stärke ihrer One-vs-Rest-Koeffizienten."""
00079
00080     classifier = OneVsRestClassifier(LogisticRegression(
00081         C=1.0,
00082         solver="liblinear",
00083         max_iter=2000,
00084         random_state=42,
00085     ))
00086
00087     classifier.fit(
00088         X_train,
00089         y_train,
00090     )
00091
00092     # Jede Klasse besitzt ein eigenes logistisches Regressionsmodell.
00093     coefficients = np.vstack([model.coef_ for model in classifier.estimators_])
00094
00095     # Die Koeffizienten aller Klassen werden über ihre L2-Norm zusammengefasst.
00096     importance = np.sqrt(
00097         np.sum(coefficients ** 2, axis=0)
00098     )
00099
00100     return np.argsort(importance)[::-1]
00101
00102

```

4.5.3 Variable Documentation

4.5.3.1 all_features

```
dict feature_selection_mean.all_features
```

Initial value:

```

= {
    model_name: load_features(feature_dir)
    for model_name, feature_dir in FEATURE_DIRS.items()
}

```

4.5.3.2 class_map

dict feature_selection_mean.class_map

Initial value:

```
= {
    "low": 0,
    "intermediate": 1,
    "high": 2,
}
```

4.5.3.3 classifier

feature_selection_mean.classifier

Initial value:

```
= OneVsRestClassifier(LogisticRegression(
    C=1.0,
    solver="liblinear",
    max_iter=2000,
    random_state=42,
))
```

4.5.3.4 FEATURE_DIRS

dict feature_selection_mean.FEATURE_DIRS

Initial value:

```
= {
    "gigapath": ROOT / "trident_runs/HE_3models/20x_256px_0px_overlap/features_gigapath",
    "uni2": ROOT / "trident_runs/HE_3models/20x_256px_0px_overlap/features_uni_v2",
    "virchow2": ROOT / "trident_runs/HE_3models/20x_224px_0px_overlap/features_virchow2",
}
```

4.5.3.5 full_balacc

feature_selection_mean.full_balacc

Initial value:

```
= balanced_accuracy_score(
    y_test,
    predictions,
)
```

4.5.3.6 full_f1

feature_selection_mean.full_f1

Initial value:

```
= f1_score(
    y_test,
    predictions,
    average="macro",
    zero_division=0,
)
```

4.5.3.7 metric_names

dict feature_selection_mean.metric_names

Initial value:

```
= {
    "full": "full_balacc",
    "top_k": "topk_balacc",
    "full_f1": "full_f1",
    "top_k_f1": "topk_f1",
}
```

4.5.3.8 oof_results

feature_selection_mean.oof_results

Initial value:

```
= repeat_df.pivot(
    index=["repeat", "k"],
    columns="model",
    values=["full", "top_k", "full_f1", "top_k_f1"],
)
```

4.5.3.9 predictions

feature_selection_mean.predictions

Initial value:

```
= classifier.predict(
    test_scaled
)
```

4.5.3.10 ranking

feature_selection_mean.ranking

Initial value:

```
= rank_features(
    train_scaled,
    y_train,
)
```

4.5.3.11 row

dict row

Initial value:

```
= {
    "split": split_number,
    "k": k,
}
```

4.5.3.12 scores

feature_selection_mean.scores

Initial value:

```
= oof_results[
    oof_results["k"] == k
][f"{model_name}_topk_balacc"]
```

4.5.3.13 selected

feature_selection_mean.selected

Initial value:

```
= model_oof[
    (model_oof["variant"] == "top_k")
    & (model_oof["k"] == k)
]
```

4.5.3.14 selection_counts

dict feature_selection_mean.selection_counts

Initial value:

```
= {
    model_name: {
        k: np.zeros(X.shape[1], dtype=int)
        for k in K_VALUES
    }
    for model_name, X in all_features.items()
}
```

4.5.3.15 splits

feature_selection_mean.splits

Initial value:

```
= list(
    splitter.split(
        np.zeros(len(y)),
        y,
    )
)
```

4.5.3.16 splitter

feature_selection_mean.splitter

Initial value:

```
= RepeatedStratifiedKFold(
    n_splits=N_SPLITS,
    n_repeats=N_REPEATS,
    random_state=42,
)
```

4.5.3.17 summary

```
tuple feature_selection_mean.summary
```

Initial value:

```
= (
    oof_results
    .groupby("k")
    .agg({
        "gigapath_topk_balacc": ["mean", "std"],
        "uni2_topk_balacc": ["mean", "std"],
        "virchow2_topk_balacc": ["mean", "std"],
    })
)
```

4.5.3.18 test_scaled

```
dict test_scaled
```

Initial value:

```
= scaler.transform(
    X[test_idx]
)
```

4.5.3.19 train_scaled

```
dict train_scaled
```

Initial value:

```
= scaler.fit_transform(
    X[train_idx]
)
```

4.6 mean_baseline Namespace Reference

Variables

- **ROOT** = Path("/home/ubuntu/applications/ki67-thesis")
- str **LABEL_FILE** = ROOT / "data/labels/ki67_clean_20_HE.csv"
- str **RESULT_DIR** = ROOT / "results/meanpool_repeated_cv"
- **parents**
- **True**
- **exist_ok**
- dict **FEATURE_DIRS**
- int **N_SPLITS** = 5
- int **N_REPEATS** = 50
- int **RANDOM_STATE** = 42
- **labels** = pd.read_csv(LABEL_FILE)
- dict **class_map**
- **y** = labels["ki67_class"].map(class_map).to_numpy().astype(int)
- **cv**
- **splits** = list(cv.split(np.zeros(len(y)), y))
- dict **slide_features** = {}
- list **model_features** = []
- str **h5_path** = feature_dir / f"{Path(he_id).stem}.h5"

- **patch_features** = f["features"][:]
- list **fold_rows** = []
- list **repeat_rows** = []
- list **oof_frames** = []
- dict **X** = slide_features[model_name]
- list **repeat_predictions**
- list **train_balanced_accuracies**
- int **repeat** = split_number // N_SPLITS
- int **fold** = split_number % N_SPLITS
- **classifier**
- **train_predictions**
- **predictions**
- list **dummy_predictions**
- **dummy** = DummyClassifier(strategy="most_frequent")
- **fold_df** = pd.DataFrame(fold_rows)
- **ignore_index**
- **index**
- **repeat_df** = pd.DataFrame(repeat_rows)
- list **summary_rows** = []
- **model_results**
- dict **row**
- **values** = model_results[metric].to_numpy()
- **ddof**
- **summary** = pd.DataFrame(summary_rows)

4.6.1 Detailed Description

Vergleicht mean-gepoolte Slide-Features mittels wiederholter Kreuzvalidierung.

Für GigaPath, UNI v2 und Virchow2 werden die Patch-Features jedes Slides durch Mean Pooling zu einem Slide-Level-Featurevektor zusammengefasst. Anschließend werden die Ki-67-Klassen mit standardisierten Features und logistischer Regression vorhergesagt.

Die Auswertung erfolgt mit 50 Wiederholungen einer stratifizierten 5-Fold-Cross-Validation. Alle Foundation-Modelle und die Dummy-Baseline verwenden dieselben Trainings- und Testaufteilungen.

Neben den einzelnen Fold- und OOF-Ergebnissen werden die Kennzahlen jeder vollständigen Wiederholung sowie deren Zusammenfassung als CSV gespeichert.

4.6.2 Variable Documentation

4.6.2.1 class_map

dict mean_baseline.class_map

Initial value:

```
= {
    "low": 0,
    "intermediate": 1,
    "high": 2,
}
```

4.6.2.2 classifier

mean_baseline.classifier

Initial value:

```
= Pipeline([
    ("scaler", StandardScaler()),
    (
        "classifier",
        OneVsRestClassifier(LogisticRegression(
            C=1.0,
            solver="liblinear",
            dual=True,
            max_iter=2000,
        )),
    ),
])
```

4.6.2.3 cv

mean_baseline.cv

Initial value:

```
= RepeatedStratifiedKFold(
    n_splits=N_SPLITS,
    n_repeats=N_REPEATS,
    random_state=RANDOM_STATE,
)
```

4.6.2.4 dummy_predictions

list mean_baseline.dummy_predictions

Initial value:

```
= [
    np.zeros(len(y), dtype=int)
    for _ in range(N_REPEATS)
]
```

4.6.2.5 FEATURE_DIRS

dict mean_baseline.FEATURE_DIRS

Initial value:

```
= {
    "gigapath": ROOT / "trident_runs/HE_3models/20x_256px_0px_overlap/features_gigapath",
    "uni_v2": ROOT / "trident_runs/HE_3models/20x_256px_0px_overlap/features_uni_v2",
    "virchow2": ROOT / "trident_runs/HE_3models/20x_224px_0px_overlap/features_virchow2",
}
```

4.6.2.6 model_results

mean_baseline.model_results

Initial value:

```
= repeat_df[
    repeat_df["model"] == model_name
]
```

4.6.2.7 predictions

```
list predictions
```

Initial value:

```
= classifier.predict(  
    X[test_idx]  
)
```

4.6.2.8 repeat_predictions

```
list mean_baseline.repeat_predictions
```

Initial value:

```
= [  
    np.zeros(len(y), dtype=int)  
    for _ in range(N_REPEATS)  
]
```

4.6.2.9 row

```
dict mean_baseline.row
```

Initial value:

```
= {  
    "model": model_name,  
}
```

4.6.2.10 train_balanced_accuracies

```
list mean_baseline.train_balanced_accuracies
```

Initial value:

```
= [  
    []  
    for _ in range(N_REPEATS)  
]
```

4.6.2.11 train_predictions

```
mean_baseline.train_predictions
```

Initial value:

```
= classifier.predict(  
    X[train_idx]  
)
```

4.7 mean_vs_attention_MIL Namespace Reference

Classes

- class [MLPClassifier](#)
- class [MeanMLP](#)
- class [AttentionMLP](#)

Functions

- [set_seed](#) (seed)
- [train_model](#) (model, train_idx)
- [evaluate](#) (model, indices, return_predictions=False)

Variables

- **ROOT** = Path("/home/ubuntu/applications/ki67-thesis")
- str **LABEL_FILE** = ROOT / "data/labels/ki67_clean_20_HE.csv"
- str **OUT_DIR** = ROOT / "results/mean_vs_attention_mlp"
- **parents**
- **True**
- **exist_ok**
- dict [FEATURE_DIRS](#)
- str **MODEL_NAME** = "virchow2"
- int **N_REPEATS** = 50
- int **EPOCHS** = 15
- int **HIDDEN_DIM** = 32
- int **ATTENTION_DIM** = 32
- float **DROPOUT** = 0.3
- int **LR** = 1e-4
- int **WEIGHT_DECAY** = 1e-3
- int **MAX_TRAIN_PATCHES** = 1024
- int **SEED** = 42
- dict **FEATURE_DIR** = FEATURE_DIRS[MODEL_NAME]
- [device](#)
- **labels** = pd.read_csv(LABEL_FILE)
- **regex**
- dict [class_map](#)
- **y** = labels["ki67_class"].map(class_map).to_numpy()
- list **bags** = []
- **features** = ff["features"][:]
- list **INPUT_DIM** = bags[0].shape[1]
- [cv](#)
- list **results** = []
- list **oof_frames** = []
- **start**
- **mean_model** = [MeanMLP](#)(INPUT_DIM)
- **mean_balacc**
- **mean_f1**
- **mean_predictions**
- **return_predictions**
- **attention_model** = [AttentionMLP](#)(INPUT_DIM)
- **attention_balacc**
- **attention_f1**
- **attention_predictions**
- tuple **repeat** = (fold_number - 1) // 5 + 1
- tuple **fold** = (fold_number - 1) % 5 + 1
- **oof_df** = pd.concat(oof_frames, ignore_index=True)
- **index**
- list **repeat_rows** = []
- dict **row** = {"repeat": repeat}
- **method_oof** = repeat_oof[repeat_oof["method"] == method]
- **average**
- **zero_division**
- **repeat_df** = pd.DataFrame(repeat_rows)
- **values** = repeat_df[ff["{method}_{metric}"]]

4.7.1 Detailed Description

Vergleicht Mean-MIL und Attention-MIL.

Beide Verfahren sagen aus gespeicherten Features eine von drei Ki67-Klassen voraus. Mean-MIL bildet den einfachen Durchschnitt aller Ausschnitte. Attention-MIL lernt, welche Ausschnitte stärker zählen sollen. Anschließend berechnet jeweils ein kleines lernendes Modell drei Klassenwerte.

Die Prüfung erfolgt in 50 Durchläufen mit jeweils fünf Bildgruppen. Vier Gruppen dienen zum Lernen, die übrige zum Prüfen; jede Gruppe wird einmal geprüft.

Die Konsole zeigt Prüfkennzahlen aus vollständigen OOF-Durchläufen. Die einzelnen Vorhersagen und Durchlaufkennzahlen werden zusätzlich als CSV gespeichert. Accuracy ist der Anteil richtig vorhergesagter Slides. Balanced Accuracy mittelt die Trefferquoten der Klassen mit gleichem Gewicht. Macro F1 bezieht auch falsche Zuordnungen und übersehene Slides je Klasse ein.

4.7.2 Function Documentation

4.7.2.1 evaluate()

```
evaluate (
    model,
    indices,
    return_predictions = False)
```

Bewertet das Modell auf den angegebenen Slides ohne weitere Lernschritte. Verwendet bags und y aus dem Modul und prüft jedes Slide mit allen Patches. Das Modell wird in den Evaluationsmodus versetzt .

```
00286 def evaluate(model, indices, return_predictions=False):
00287     """
00288     Bewertet das Modell auf den angegebenen Slides ohne weitere Lernschritte.
00289     Verwendet bags und y aus dem Modul und prüft jedes Slide mit allen
00290     Patches. Das Modell wird in den Evaluationsmodus versetzt .
00291     """
00292
00293     model.eval()
00294
00295     predictions = []
00296     targets = []
00297
00298     # Bei der Evaluation werden keine Gradienten benötigt.
00299     with torch.no_grad():
00300
00301         for idx in indices:
00302
00303             x = bags[idx].to(device)
00304
00305             logits = model(x)
00306
00307             prediction = torch.argmax(
00308                 logits
00309             ).item()
00310
00311             predictions.append(prediction)
00312             targets.append(y[idx])
00313
00314     balanced_acc = balanced_accuracy_score(
00315         targets,
00316         predictions,
00317     )
00318
00319     macro_f1 = f1_score(
00320         targets,
00321         predictions,
00322         average="macro",
00323         zero_division=0,
00324     )
```

```

00325
00326     if return_predictions:
00327         return balanced_acc, macro_f1, np.asarray(predictions, dtype=int)
00328
00329     return balanced_acc, macro_f1
00330
00331

```

4.7.2.2 set_seed()

```

set_seed (
    seed)

```

Setzt die Seeds der verwendeten Zufallsgeneratoren.

```

00094 def set_seed(seed):
00095     """
00096     Setzt die Seeds der verwendeten Zufallsgeneratoren.
00097     """
00098
00099     random.seed(seed)
00100     np.random.seed(seed)
00101     torch.manual_seed(seed)
00102
00103     if torch.cuda.is_available():
00104         torch.cuda.manual_seed_all(seed)
00105
00106

```

4.7.2.3 train_model()

```

train_model (
    model,
    train_idx)

```

Trainiert das übergebene Modell an den angegebenen Slides.

Verwendet die global geladenen Merkmale bags, die Klassen y und die Einstellungen für Trainingsdauer, Schrittgröße und Rechengegerät. Die Gewichte des übergebenen Modells werden direkt verändert.

```

00226 def train_model(model, train_idx):
00227     """
00228     Trainiert das übergebene Modell an den angegebenen Slides.
00229
00230     Verwendet die global geladenen Merkmale bags, die Klassen y und
00231     die Einstellungen für Trainingsdauer, Schrittgröße und Rechengegerät.
00232     Die Gewichte des übergebenen Modells werden direkt verändert.
00233     """
00234
00235     model = model.to(device)
00236
00237     optimizer = torch.optim.AdamW(
00238         model.parameters(),
00239         lr=LR,
00240         weight_decay=WEIGHT_DECAY,
00241     )
00242
00243     for _ in range(EPOCHS):
00244
00245         model.train()
00246
00247         for idx in np.random.permutation(train_idx):
00248
00249             x = bags[idx]
00250
00251             # Bei vielen Ausschnitten wird bei jedem Besuch eine neue zufällige Auswahl genutzt. Kein
00252             Ausschnitt wird dabei mehrfach ausgewählt.
00253             if len(x) > MAX_TRAIN_PATCHES:
00254
00255                 selected = np.random.choice(

```

```

00255             len(x),
00256             MAX_TRAIN_PATCHES,
00257             replace=False,
00258         )
00259
00260         x = x[selected]
00261
00262         x = x.to(device)
00263
00264         target = torch.tensor(
00265             [y[idx]],
00266             dtype=torch.long,
00267             device=device,
00268         )
00269
00270         # Vor jedem Optimierungsschritt werden die Gradienten zurückgesetzt.
00271         optimizer.zero_grad()
00272
00273         logits = model(x)
00274
00275         loss = nn.functional.cross_entropy(
00276             logits.unsqueeze(0),
00277             target,
00278         )
00279
00280         loss.backward()
00281         optimizer.step()
00282
00283     return model
00284
00285

```

4.7.3 Variable Documentation

4.7.3.1 class_map

dict mean_vs_attention_MIL.class_map

Initial value:

```

= {
    "low": 0,
    "intermediate": 1,
    "high": 2,
}

```

4.7.3.2 cv

mean_vs_attention_MIL.cv

Initial value:

```

= RepeatedStratifiedKFold(
    n_splits=5,
    n_repeats=N_REPEATS,
    random_state=SEED,
)

```

4.7.3.3 device

mean_vs_attention_MIL.device

Initial value:

```

= torch.device(
    "cuda" if torch.cuda.is_available() else "cpu"
)

```

4.7.3.4 FEATURE_DIRS

dict mean_vs_attention_MIL.FEATURE_DIRS

Initial value:

```
= {  
    "gigapath": ROOT / "trident_runs/HE_3models/20x_256px_0px_overlap/features_gigapath",  
    "uni_v2": ROOT / "trident_runs/HE_3models/20x_256px_0px_overlap/features_uni_v2",  
    "virchow2": ROOT / "trident_runs/HE_3models/20x_224px_0px_overlap/features_virchow2",  
}
```

Chapter 5

Class Documentation

5.1 mean_vs_attention_MIL.AttentionMLP Class Reference

Inheritance diagram for mean_vs_attention_MIL.AttentionMLP:

Collaboration diagram for mean_vs_attention_MIL.AttentionMLP:

Public Member Functions

- `__init__` (self, input_dim)
- `forward` (self, x)

Public Attributes

- `attention_v`
- `attention_u`
- `attention_w`
- `classifier`

5.1.1 Detailed Description

Lernt die Gewichtung der Patches und sagt daraus die Bildklasse voraus.

Attributes:

```
attention_v (nn.Linear): Lineare Projektion für Tanh-Zweig.  
attention_u (nn.Linear): Lineare Projektion für Sigmoid-Zweig.  
attention_w (nn.Linear): Berechnet einen Attentionsscore je Patch.  
classifier (MLPClassifier): Klassifiziert das aggregierte Slide-Embedding.
```

5.1.2 Constructor & Destructor Documentation

5.1.2.1 `__init__()`

```
mean_vs_attention_MIL.AttentionMLP.__init__ (
    self,
    input_dim)
```

Legt die lernbare Gewichtung und die Klassenvorhersage an.

```
00167     def __init__(self, input_dim):
00168         """
00169         Legt die lernbare Gewichtung und die Klassenvorhersage an.
00170         """
00171
00172         super().__init__()
00173
00174         self.attention_v = nn.Linear(
00175             input_dim,
00176             ATTENTION_DIM,
00177         )
00178
00179         self.attention_u = nn.Linear(
00180             input_dim,
00181             ATTENTION_DIM,
00182         )
00183
00184         self.attention_w = nn.Linear(
00185             ATTENTION_DIM,
00186             1,
00187         )
00188
00189         self.classifier = MLPClassifier(
00190             input_dim
00191         )
00192
```

5.1.3 Member Function Documentation

5.1.3.1 `forward()`

```
mean_vs_attention_MIL.AttentionMLP.forward (
    self,
    x)
```

Fasst Patches mit gelernten Gewichten zusammen und bewertet Klassen.

```
00193     def forward(self, x):
00194         """
00195         Fasst Patches mit gelernten Gewichten zusammen und bewertet Klassen.
00196         """
00197
00198         a_v = torch.tanh(
00199             self.attention_v(x)
00200         )
00201
00202         a_u = torch.sigmoid(
00203             self.attention_u(x)
00204         )
00205
00206         scores = self.attention_w(
00207             a_v * a_u
00208         ).squeeze(1)
00209
00210         # Softmax erzeugt positive und negative Gewichte mit Summe 1. So entsteht ein gewichteter
00211         Durchschnitt der Patch Features.
00212         weights = torch.softmax(
00213             scores,
00214             dim=0,
```

```
00215
00216         slide_embedding = torch.sum(
00217             weights.unsqueeze(1) * x,
00218             dim=0,
00219         )
00220
00221         return self.classifier(
00222             slide_embedding
00223         )
00224
00225
```

5.1.4 Member Data Documentation

5.1.4.1 attention_u

mean_vs_attention_MIL.AttentionMLP.attention_u

Initial value:

```
= nn.Linear(
    input_dim,
    ATTENTION_DIM,
)
```

5.1.4.2 attention_v

mean_vs_attention_MIL.AttentionMLP.attention_v

Initial value:

```
= nn.Linear(
    input_dim,
    ATTENTION_DIM,
)
```

5.1.4.3 attention_w

mean_vs_attention_MIL.AttentionMLP.attention_w

Initial value:

```
= nn.Linear(
    ATTENTION_DIM,
    1,
)
```

5.1.4.4 classifier

mean_vs_attention_MIL.AttentionMLP.classifier

Initial value:

```
= MLPClassifier(
    input_dim
)
```

The documentation for this class was generated from the following file:

- mean_vs_attention_MIL.py

5.2 CNN_patient_split.CellDataset Class Reference

Inheritance diagram for CNN_patient_split.CellDataset:

Collaboration diagram for CNN_patient_split.CellDataset:

Public Member Functions

- [__init__](#) (self, paths, labels, transform)
- [__len__](#) (self)
- [__getitem__](#) (self, index)

Public Attributes

- **paths** = paths
- **labels** = labels
- **transform** = transform

5.2.1 Detailed Description

Lädt einzelne Zellbilder und ihre binären Ki-67-Labels.

5.2.2 Constructor & Destructor Documentation

5.2.2.1 __init__()

```
CNN_patient_split.CellDataset.__init__ (
    self,
    paths,
    labels,
    transform)
00151     def __init__(self, paths, labels, transform):
00152         self.paths = paths
00153         self.labels = labels
00154         self.transform = transform
00155
```

5.2.3 Member Function Documentation

5.2.3.1 __getitem__()

```
CNN_patient_split.CellDataset.__getitem__ (
    self,
    index)
00159     def __getitem__(self, index):
00160
00161         image = Image.open(self.paths[index]).convert("RGB")
00162         image = self.transform(image)
00163
00164         label = torch.tensor(
00165             self.labels[index],
00166             dtype=torch.float32,
00167         )
00168
00169         return image, label
00170
00171
```

5.2.3.2 `__len__()`

```
CNN_patient_split.CellDataset.__len__ (  
    self)  
00156     def __len__(self):  
00157         return len(self.paths)  
00158
```

The documentation for this class was generated from the following file:

- CNN_patient_split.py

5.3 CNN_random_split.CellDataset Class Reference

Inheritance diagram for CNN_random_split.CellDataset:

Collaboration diagram for CNN_random_split.CellDataset:

Public Member Functions

- [__init__](#) (self, paths, labels, transform)
- [__len__](#) (self)
- [__getitem__](#) (self, index)

Public Attributes

- **paths** = paths
- **labels** = labels
- **transform** = transform

5.3.1 Detailed Description

Lädt einzelne Zellbilder und ihre binären Ki-67-Labels.

5.3.2 Constructor & Destructor Documentation

5.3.2.1 `__init__()`

```
CNN_random_split.CellDataset.__init__ (  
    self,  
    paths,  
    labels,  
    transform)  
00137     def __init__(self, paths, labels, transform):  
00138         self.paths = paths  
00139         self.labels = labels  
00140         self.transform = transform  
00141
```

5.3.3 Member Function Documentation

5.3.3.1 `__getitem__()`

```
CNN_random_split.CellDataset.__getitem__ (
    self,
    index)
00145     def __getitem__(self, index):
00146
00147         image = Image.open(self.paths[index]).convert("RGB")
00148         image = self.transform(image)
00149
00150         label = torch.tensor(
00151             self.labels[index],
00152             dtype=torch.float32,
00153         )
00154
00155         return image, label
00156
00157
```

5.3.3.2 `__len__()`

```
CNN_random_split.CellDataset.__len__ (
    self)
00142     def __len__(self):
00143         return len(self.paths)
00144
```

The documentation for this class was generated from the following file:

- CNN_random_split.py

5.4 mean_vs_attention_MIL.MeanMLP Class Reference

Inheritance diagram for mean_vs_attention_MIL.MeanMLP:

Collaboration diagram for mean_vs_attention_MIL.MeanMLP:

Public Member Functions

- `__init__` (self, input_dim)
- `forward` (self, x)

Public Attributes

- `classifier` = `MLPClassifier`(input_dim)

5.4.1 Detailed Description

Gewichtet alle Patches gleich und sagt daraus die Bildklasse voraus.

5.4.2 Constructor & Destructor Documentation

5.4.2.1 __init__()

```
mean_vs_attention_MIL.MeanMLP.__init__ (
    self,
    input_dim)
```

Erstellt das Modell für gemittelte Patch Features.

```
00139     def __init__(self, input_dim):
00140         """Erstellt das Modell für gemittelte Patch Features.
00141         """
00142
00143         super().__init__()
00144         self.classifier = MLPClassifier(input_dim)
00145
```

5.4.3 Member Function Documentation

5.4.3.1 forward()

```
mean_vs_attention_MIL.MeanMLP.forward (
    self,
    x)
```

Mittelt die Patch Features und berechnet drei Klassenwerte.

```
00146     def forward(self, x):
00147         """Mittelt die Patch Features und berechnet drei Klassenwerte.
00148         """
00149
00150         slide_embedding = x.mean(dim=0)
00151
00152         return self.classifier(
00153             slide_embedding
00154         )
00155
00156
```

The documentation for this class was generated from the following file:

- mean_vs_attention_MIL.py

5.5 mean_vs_attention_MIL.MLPClassifier Class Reference

Inheritance diagram for mean_vs_attention_MIL.MLPClassifier:

Collaboration diagram for mean_vs_attention_MIL.MLPClassifier:

Public Member Functions

- [__init__](#) (self, input_dim)
- [forward](#) (self, x)

Public Attributes

- [layers](#)

5.5.1 Detailed Description

Berechnet aus den Merkmalen eines Slides drei Klassenwerte.

5.5.2 Constructor & Destructor Documentation

5.5.2.1 `__init__()`

```
mean_vs_attention_MIL.MLPClassifier.__init__ (  
    self,  
    input_dim)
```

Legt die Schichten den Klassifikationsmodelles an.

```
00112     def __init__(self, input_dim):  
00113         """  
00114         Legt die Schichten den Klassifikationsmodelles an.  
00115         """  
00116  
00117         super().__init__()  
00118  
00119         self.layers = nn.Sequential(  
00120             nn.Linear(input_dim, HIDDEN_DIM),  
00121             nn.ReLU(),  
00122             nn.Dropout(DROPOUT),  
00123             nn.Linear(HIDDEN_DIM, 3),  
00124         )  
00125
```

5.5.3 Member Function Documentation

5.5.3.1 `forward()`

```
mean_vs_attention_MIL.MLPClassifier.forward (  
    self,  
    x)
```

Berechnet die Klassenwerte aus bereits zusammengefassten Features.

```
00126     def forward(self, x):  
00127         """  
00128         Berechnet die Klassenwerte aus bereits zusammengefassten Features.  
00129         """  
00130  
00131         return self.layers(x)  
00132  
00133
```

5.5.4 Member Data Documentation

5.5.4.1 layers

mean_vs_attention_MIL.MLPClassifier.layers

Initial value:

```
= nn.Sequential(  
    nn.Linear(input_dim, HIDDEN_DIM),  
    nn.ReLU(),  
    nn.Dropout(DROPOUT),  
    nn.Linear(HIDDEN_DIM, 3),  
)
```

The documentation for this class was generated from the following file:

- mean_vs_attention_MIL.py

Index

- `__getitem__`
 - CNN_patient_split.CellDataset, [46](#)
 - CNN_random_split.CellDataset, [48](#)
 - `__init__`
 - CNN_patient_split.CellDataset, [46](#)
 - CNN_random_split.CellDataset, [47](#)
 - mean_vs_attention_MIL.AttentionMLP, [44](#)
 - mean_vs_attention_MIL.MeanMLP, [49](#)
 - mean_vs_attention_MIL.MLPClassifier, [50](#)
 - `__len__`
 - CNN_patient_split.CellDataset, [46](#)
 - CNN_random_split.CellDataset, [48](#)
- accuracy
 - CNN_patient_split, [9](#)
 - CNN_random_split, [15](#)
- AGGREGATIONS
 - compare_aggregations, [20](#)
- all_features
 - feature_selection_aggregations, [25](#)
 - feature_selection_mean, [30](#)
- attention_u
 - mean_vs_attention_MIL.AttentionMLP, [45](#)
- attention_v
 - mean_vs_attention_MIL.AttentionMLP, [45](#)
- attention_w
 - mean_vs_attention_MIL.AttentionMLP, [45](#)
- auc
 - CNN_patient_split, [9](#)
 - CNN_random_split, [15](#)
- balanced_accuracy
 - CNN_patient_split, [9](#)
 - CNN_random_split, [15](#)
- class_map
 - compare_aggregations, [20](#)
 - feature_selection_aggregations, [25](#)
 - feature_selection_mean, [30](#)
 - mean_baseline, [35](#)
 - mean_vs_attention_MIL, [41](#)
- classifier
 - compare_aggregations, [20](#)
 - feature_selection_aggregations, [25](#)
 - feature_selection_mean, [31](#)
 - mean_baseline, [35](#)
 - mean_vs_attention_MIL.AttentionMLP, [45](#)
- CNN_patient_split, [7](#)
 - accuracy, [9](#)
 - auc, [9](#)
 - balanced_accuracy, [9](#)
 - criterion, [9](#)
 - DATA_DIR, [9](#)
 - DEVICE, [9](#)
 - eval_transform, [10](#)
 - model, [10](#)
 - optimizer, [10](#)
 - predictions, [10](#)
 - RESULT_DIR, [10](#)
 - splitter, [11](#)
 - test_dataset, [11](#)
 - test_loader, [11](#)
 - train_accuracy, [11](#)
 - train_dataset, [11](#)
 - train_loader, [12](#)
 - train_transform, [12](#)
 - val_accuracy, [12](#)
 - val_dataset, [12](#)
 - val_loader, [12](#)
- CNN_patient_split.CellDataset, [46](#)
 - `__getitem__`, [46](#)
 - `__init__`, [46](#)
 - `__len__`, [46](#)
- CNN_random_split, [13](#)
 - accuracy, [15](#)
 - auc, [15](#)
 - balanced_accuracy, [15](#)
 - criterion, [15](#)
 - DATA_DIR, [15](#)
 - DEVICE, [15](#)
 - eval_transform, [16](#)
 - model, [16](#)
 - optimizer, [16](#)
 - predictions, [16](#)
 - RESULT_DIR, [16](#)
 - test_dataset, [17](#)
 - test_loader, [17](#)
 - train_accuracy, [17](#)
 - train_dataset, [17](#)
 - train_loader, [17](#)
 - train_transform, [18](#)
 - val_accuracy, [18](#)
 - val_dataset, [18](#)
 - val_loader, [18](#)
- CNN_random_split.CellDataset, [47](#)
 - `__getitem__`, [48](#)
 - `__init__`, [47](#)
 - `__len__`, [48](#)
- compare_aggregations, [19](#)

- AGGREGATIONS, 20
 - class_map, 20
 - classifier, 20
 - cv, 20
 - FEATURE_DIRS, 20
 - mean, 20
 - model_features, 21
 - model_results, 21
 - predictions, 21
 - repeat_predictions, 21
 - row, 21
 - std, 22
- criterion
 - CNN_patient_split, 9
 - CNN_random_split, 15
- cv
 - compare_aggregations, 20
 - mean_baseline, 36
 - mean_vs_attention_MIL, 41
- DATA_DIR
 - CNN_patient_split, 9
 - CNN_random_split, 15
- DEVICE
 - CNN_patient_split, 9
 - CNN_random_split, 15
- device
 - mean_vs_attention_MIL, 41
- dummy_predictions
 - mean_baseline, 36
- eval_transform
 - CNN_patient_split, 10
 - CNN_random_split, 16
- evaluate
 - mean_vs_attention_MIL, 39
- FEATURE_DIRS
 - compare_aggregations, 20
 - feature_selection_aggregations, 25
 - feature_selection_mean, 31
 - mean_baseline, 36
 - mean_vs_attention_MIL, 41
- feature_selection_aggregations, 22
 - all_features, 25
 - class_map, 25
 - classifier, 25
 - FEATURE_DIRS, 25
 - full_score, 25
 - load_features, 23
 - model_results, 25
 - predictions, 26
 - rank_dimensions, 24
 - ranking, 26
 - row, 26
 - scores, 26
 - selected, 26
 - splits, 26
 - splitter, 27
 - test_3d, 27
 - test_top_k, 27
 - top_k_score, 27
 - train_3d, 27
 - train_top_k, 28
- feature_selection_mean, 28
 - all_features, 30
 - class_map, 30
 - classifier, 31
 - FEATURE_DIRS, 31
 - full_balacc, 31
 - full_f1, 31
 - load_features, 29
 - metric_names, 31
 - oof_results, 32
 - predictions, 32
 - rank_features, 30
 - ranking, 32
 - row, 32
 - scores, 32
 - selected, 33
 - selection_counts, 33
 - splits, 33
 - splitter, 33
 - summary, 33
 - test_scaled, 34
 - train_scaled, 34
- forward
 - mean_vs_attention_MIL.AttentionMLP, 44
 - mean_vs_attention_MIL.MeanMLP, 49
 - mean_vs_attention_MIL.MLPClassifier, 50
- full_balacc
 - feature_selection_mean, 31
- full_f1
 - feature_selection_mean, 31
- full_score
 - feature_selection_aggregations, 25
- layers
 - mean_vs_attention_MIL.MLPClassifier, 51
- load_features
 - feature_selection_aggregations, 23
 - feature_selection_mean, 29
- mean
 - compare_aggregations, 20
- mean_baseline, 34
 - class_map, 35
 - classifier, 35
 - cv, 36
 - dummy_predictions, 36
 - FEATURE_DIRS, 36
 - model_results, 36
 - predictions, 36
 - repeat_predictions, 37
 - row, 37
 - train_balanced_accuracies, 37
 - train_predictions, 37
- mean_vs_attention_MIL, 37

- class_map, 41
- cv, 41
- device, 41
- evaluate, 39
- FEATURE_DIRS, 41
- set_seed, 40
- train_model, 40
- mean_vs_attention_MIL.AttentionMLP, 43
 - __init__, 44
 - attention_u, 45
 - attention_v, 45
 - attention_w, 45
 - classifier, 45
 - forward, 44
- mean_vs_attention_MIL.MeanMLP, 48
 - __init__, 49
 - forward, 49
- mean_vs_attention_MIL.MLPClassifier, 49
 - __init__, 50
 - forward, 50
 - layers, 51
- metric_names
 - feature_selection_mean, 31
- model
 - CNN_patient_split, 10
 - CNN_random_split, 16
- model_features
 - compare_aggregations, 21
- model_results
 - compare_aggregations, 21
 - feature_selection_aggregations, 25
 - mean_baseline, 36
- oof_results
 - feature_selection_mean, 32
- optimizer
 - CNN_patient_split, 10
 - CNN_random_split, 16
- predictions
 - CNN_patient_split, 10
 - CNN_random_split, 16
 - compare_aggregations, 21
 - feature_selection_aggregations, 26
 - feature_selection_mean, 32
 - mean_baseline, 36
- rank_dimensions
 - feature_selection_aggregations, 24
- rank_features
 - feature_selection_mean, 30
- ranking
 - feature_selection_aggregations, 26
 - feature_selection_mean, 32
- repeat_predictions
 - compare_aggregations, 21
 - mean_baseline, 37
- RESULT_DIR
 - CNN_patient_split, 10
- CNN_random_split, 16
- row
 - compare_aggregations, 21
 - feature_selection_aggregations, 26
 - feature_selection_mean, 32
 - mean_baseline, 37
- scores
 - feature_selection_aggregations, 26
 - feature_selection_mean, 32
- selected
 - feature_selection_aggregations, 26
 - feature_selection_mean, 33
- selection_counts
 - feature_selection_mean, 33
- set_seed
 - mean_vs_attention_MIL, 40
- splits
 - feature_selection_aggregations, 26
 - feature_selection_mean, 33
- splitter
 - CNN_patient_split, 11
 - feature_selection_aggregations, 27
 - feature_selection_mean, 33
- std
 - compare_aggregations, 22
- summary
 - feature_selection_mean, 33
- test_3d
 - feature_selection_aggregations, 27
- test_dataset
 - CNN_patient_split, 11
 - CNN_random_split, 17
- test_loader
 - CNN_patient_split, 11
 - CNN_random_split, 17
- test_scaled
 - feature_selection_mean, 34
- test_top_k
 - feature_selection_aggregations, 27
- top_k_score
 - feature_selection_aggregations, 27
- train_3d
 - feature_selection_aggregations, 27
- train_accuracy
 - CNN_patient_split, 11
 - CNN_random_split, 17
- train_balanced_accuracies
 - mean_baseline, 37
- train_dataset
 - CNN_patient_split, 11
 - CNN_random_split, 17
- train_loader
 - CNN_patient_split, 12
 - CNN_random_split, 17
- train_model
 - mean_vs_attention_MIL, 40
- train_predictions

- mean_baseline, [37](#)
- train_scaled
 - feature_selection_mean, [34](#)
- train_top_k
 - feature_selection_aggregations, [28](#)
- train_transform
 - CNN_patient_split, [12](#)
 - CNN_random_split, [18](#)
- val_accuracy
 - CNN_patient_split, [12](#)
 - CNN_random_split, [18](#)
- val_dataset
 - CNN_patient_split, [12](#)
 - CNN_random_split, [18](#)
- val_loader
 - CNN_patient_split, [12](#)
 - CNN_random_split, [18](#)